Predicción de volatilidad financiera

Semana 9: modelos base de volatilidad histórica y media móvil

Objetivo. Construir pronósticos sencillos de la volatilidad futura a cinco sesiones y establecer referencias reproducibles de desempeño. Se utilizará la base de la Semana 8, respetando la disponibilidad temporal de la información y comparando todos los modelos sobre las mismas observaciones.

Índice

1. Continuidad con el proyecto **3**

2. Objetivos de aprendizaje **3**

3. Qué es un modelo base **3**

4. Modelo de volatilidad histórica **4**

4.1. Unidades y anualización 4

5. Modelo de media móvil de volatilidades **4**

5.1. Tres operaciones distintas 5

6. Ejemplo numérico resuelto **5**

7. Disponibilidad temporal y fuga de información **5**

8. Organización del trabajo computacional **6**

9. Verificación del objetivo de Semana 8 **7**

10. Construcción de las predicciones **7**

11. Comprobaciones manuales de los modelos **8**

12. Muestra común y valores faltantes **8**

13. Evaluación preliminar en un bloque final **9**

13.1. Objetivos que atraviesan la frontera 9

14. Métricas de error **10**

14.1. MAPE y objetivos pequeños 10

15. Cálculo de la tabla comparativa **10**

16. Gráficas e interpretación **11**

17. Solapamiento de ventanas futuras **12**

18. Exportación y trazabilidad **12**

19. Productos esperados y conexión con la Semana 10 **13**

1. Continuidad con el proyecto

En la Semana 8 se construyeron las variables `Target_Vol_5`, `Target_Vol_10` y `Target_Vol_20`. La entrada de esta semana es

`data/processed/SPY_2015_2025_targets.csv`.

El cronograma del primer proyecto asigna a la Semana 9 la implementación de modelos base de volatilidad histórica y media móvil. EWMA corresponde a la Semana 10. Por ello, aquí se establecen referencias sencillas; no se ajustan todavía GARCH ni modelos de aprendizaje automático.

Se toma como objetivo principal

$$Y_t = RV_{t,5} = \sqrt{\frac{1}{4}\sum_{j=1}^{5}(r_{t+j} - \bar{r}^F_{t,5})^2}, \qquad \bar{r}^F_{t,5} = \frac{1}{5}\sum_{j=1}^{5} r_{t+j}.$$

La predicción se emite después del cierre de la sesión t. El objetivo usa los rendimientos de las cinco sesiones siguientes. En todo el documento, "días" significa sesiones bursátiles consecutivas en la base, no días naturales.

2. Objetivos de aprendizaje

Al finalizar, la estudiante deberá ser capaz de:

1. distinguir una variable observada de un pronóstico;
2. explicar qué es un modelo base y por qué es necesario;
3. implementar la persistencia de volatilidad histórica;
4. distinguir la media móvil de rendimientos de la media móvil de volatilidades;
5. verificar que una predicción sólo utiliza información hasta t;
6. construir una muestra común de evaluación;
7. interpretar MAE, RMSE, sesgo y las limitaciones de MAPE;
8. documentar resultados sin confundir un análisis preliminar con una comparación final;
9. guardar predicciones, métricas y figuras reproducibles.

3. Qué es un modelo base

Un modelo base es una regla simple que produce predicciones y permite medir si un método más elaborado aporta mejoras. Su utilidad no depende de ser el modelo con menor error: proporciona una referencia transparente y fácil de reproducir.

La persistencia supone que la volatilidad reciente es informativa sobre la volatilidad próxima. No implica que ambas sean idénticas ni que el mercado sea estacionario. Los cambios bruscos de régimen pueden hacer que la regla reaccione con retraso.

Pronóstico frente a medición. $\sigma_{t,5}$ es una medida retrospectiva conocida al cierre de t. Cuando se utiliza como estimación de Y_t, pasa a ser el pronóstico $\widehat{Y}_t = \sigma_{t,5}$. El valor observado Y_t sólo se conoce al cierre de $t + 5$.

4. Modelo de volatilidad histórica

Para $w \in \{5, 10, 20, 30\}$, la Semana 6 definió

$$\bar{r}_{t,w} = \frac{1}{w} \sum_{j=0}^{w-1} r_{t-j}, \qquad \sigma_{t,w} = \sqrt{\frac{1}{w-1} \sum_{j=0}^{w-1} (r_{t-j} - \bar{r}_{t,w})^2}.$$

La regla de pronóstico es

$$\boxed{\widehat{Y}_t^{\mathrm{HV}(w)} = \sigma_{t,w}.}$$

Se construyen cuatro referencias: HV5, HV10, HV20 y HV30. El horizonte objetivo permanece fijo en cinco sesiones; w controla la memoria retrospectiva, no el horizonte futuro.

Una ventana corta puede responder rápidamente a cambios recientes, aunque fluctúa más. Una ventana larga produce estimaciones más suaves y puede reaccionar más lentamente. Estas son tendencias generales, no garantías sobre qué ventana tendrá menor error.

4.1. Unidades y anualización

Si los rendimientos son decimales, 0.01 equivale a un rendimiento del uno por ciento. Una volatilidad de 0.012 corresponde a una desviación estándar diaria del 1.2 %. La variable objetivo es una desviación estándar de rendimientos diarios medida en una ventana de cinco sesiones; no es la volatilidad del rendimiento acumulado a cinco sesiones. No debe multiplicarse por $\sqrt{5}$ para compararla con `Vol_5`.

En esta semana se trabaja sin anualizar. Si en otra etapa se adopta la convención $\sqrt{252}$, debe aplicarse tanto al objetivo como a cada pronóstico. MAE y RMSE cambiarían por ese mismo factor; no cambiaría su orden entre modelos.

5. Modelo de media móvil de volatilidades

Aquí "media móvil" significa el promedio de estimaciones recientes de volatilidad histórica, no el promedio de los rendimientos ni de los precios. Para una ventana histórica w y un suavizamiento

k,

$$\boxed{\widehat{Y}_t^{\mathrm{MA}(w,k)} = \frac{1}{k}\sum_{j=0}^{k-1} \sigma_{t-j,w}.}$$

Se fijan previamente $w = 5$ y $k \in \{5, 10, 20\}$. Los nombres serán MA5, MA10 y MA20; todos suavizan `Vol_5`.

Por ejemplo,

$$\widehat{Y}_t^{\mathrm{MA}(5,5)} = \frac{\sigma_{t,5} + \sigma_{t-1,5} + \sigma_{t-2,5} + \sigma_{t-3,5} + \sigma_{t-4,5}}{5}.$$

La primera volatilidad usa $r_{t-4}, \ldots, r_t$ y la última usa $r_{t-8}, \ldots, r_{t-4}$. En conjunto interviene información desde $t-8$ hasta t, con ventanas solapadas.

5.1. Tres operaciones distintas

$$\mathrm{sd}(r_{t-k+1}, \ldots, r_t), \qquad \frac{1}{k}\sum_{j=0}^{k-1} \sigma_{t-j,5}, \qquad \sqrt{\frac{1}{k}\sum_{j=0}^{k-1} \sigma_{t-j,5}^2}$$

son, respectivamente, volatilidad histórica, promedio de volatilidades y raíz del promedio de varianzas. No son intercambiables. En esta semana se utiliza la segunda para los modelos MA.

Asimismo, la media de rendimientos estima una localización, no su dispersión: una serie con cambios de signos y grandes oscilaciones puede tener media cercana a cero y volatilidad alta.

6. Ejemplo numérico resuelto

Supóngase que las últimas cinco estimaciones de `Vol_5`, en orden temporal, son

$$0.010,\ 0.012,\ 0.011,\ 0.014,\ 0.013.$$

La persistencia HV5 pronostica 0.013. La regla MA5 produce

$$\widehat{Y}_t^{\mathrm{MA5}} = \frac{0.010 + 0.012 + 0.011 + 0.014 + 0.013}{5} = 0.012.$$

Si posteriormente se observa $Y_t = 0.015$, sus errores, definidos como observado menos predicho, son 0.002 y 0.003. En esta fecha HV5 tiene menor error absoluto. Una sola observación no determina el desempeño general.

7. Disponibilidad temporal y fuga de información

Los pronósticos anteriores incluyen información de t porque se emiten después del cierre. Para pronosticar antes del cierre habría que modificar el conjunto de información y desplazar las variables según su disponibilidad.

Son errores metodológicos:

- usar `Target_Vol_5` como predictor;
- usar una ventana centrada, porque incorpora observaciones posteriores;
- utilizar desplazamientos negativos para construir pronósticos;
- rellenar volatilidades iniciales con valores futuros;
- elegir la ventana de menor error en prueba y luego reportar ese error como si la elección hubiera sido previa.

No hay fuga por guardar el objetivo junto a los predictores, siempre que se mantenga explícita su función de respuesta. Los modelos de esta semana no usan columnas objetivo para pronosticar.

8. Organización del trabajo computacional

El notebook previsto se llamará `09_modelos_base.ipynb`. Los siguientes bloques se ejecutan en orden desde la raíz del proyecto. No requieren descargar datos nuevos ni bibliotecas de aprendizaje automático.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

ruta = Path("data/processed")
entrada = ruta / "SPY_2015_2025_targets.csv"
salida = Path("results/semana_09")
salida.mkdir(parents=True, exist_ok=True)

if not entrada.exists():
    raise FileNotFoundError(entrada)
datos = pd.read_csv(entrada, index_col=0,
                    parse_dates=True)
assert isinstance(datos.index, pd.DatetimeIndex)
assert not datos.index.isna().any()
assert datos.index.is_unique
assert datos.index.is_monotonic_increasing

necesarias = ["Return", "Vol_5", "Vol_10",
              "Vol_20", "Vol_30", "Target_Vol_5"]
faltantes = [c for c in necesarias
             if c not in datos.columns]
assert not faltantes, f"Faltan columnas: {faltantes}"
assert len(datos) > 100
assert np.isfinite(datos["Return"]).all()
for c in necesarias[1:]:
    s = datos[c].dropna()
    assert len(s) > 0
    assert np.isfinite(s).all()
    assert (s >= 0).all()
print(datos.shape, datos.index.min(), datos.index.max())
```

No se ordena ni elimina silenciosamente una fecha duplicada: si estas verificaciones fallan, se corrige el problema en la etapa correspondiente.

9. Verificación del objetivo de Semana 8

Conservamos la definición y comprobamos su alineación. Sólo se usa el objetivo principal; los últimos diez o veinte objetivos faltantes de otros horizontes no deben eliminar observaciones de esta comparación.

```
h = 5
objetivo = "Target_Vol_5"
y = datos[objetivo]
assert y.iloc[-h:].isna().all()
assert y.iloc[:-h].notna().all()
assert y.isna().sum() == h

esperado = (datos["Return"].rolling(h)
            .std(ddof=1).shift(-h))
np.testing.assert_allclose(
    y.to_numpy(), esperado.to_numpy(),
    rtol=1e-7, atol=1e-10, equal_nan=True)
```

Las tolerancias permiten pequeñas diferencias por exportación a CSV. No deben aumentarse para ocultar errores de alineación.

10. Construcción de las predicciones

```
pred = pd.DataFrame(index=datos.index)
for w in [5, 10, 20, 30]:
    pred[f"HV{w}"] = datos[f"Vol_{w}"]
for k in [5, 10, 20]:
    pred[f"MA{k}"] = (
        datos["Vol_5"].rolling(k, min_periods=k).mean()
    )
modelos = pred.columns.tolist()
assert not any("Target" in c for c in modelos)
for c in modelos:
    assert np.isfinite(pred[c].dropna()).all()
    assert (pred[c].dropna() >= 0).all()
```

HV reutiliza las medidas ya calculadas. MA exige k volatilidades disponibles. No se usa `center=True`, ni `shift(-1)`, ni relleno hacia atrás.

11. Comprobaciones manuales de los modelos

Para HV, verificamos directamente los rendimientos. Para MA, comprobamos el promedio de las volatilidades correspondientes.

```
i = 80
for w in [5, 10, 20, 30]:
    ventana = datos["Return"].iloc[i-w+1:i+1]
    assert len(ventana) == w
    np.testing.assert_allclose(
        pred[f"HV{w}"].iloc[i],
        ventana.std(ddof=1), rtol=1e-7, atol=1e-10)
for k in [5, 10, 20]:
    ventana = datos["Vol_5"].iloc[i-k+1:i+1]
    assert len(ventana) == k
    assert ventana.notna().all()
    np.testing.assert_allclose(
        pred[f"MA{k}"].iloc[i], ventana.mean(),
        rtol=1e-7, atol=1e-10)
```

Si una ventana de volatilidad histórica usa una convención distinta de `ddof=1`, debe corregirse o documentarse antes de comparar.

12. Muestra común y valores faltantes

Los faltantes iniciales se deben a las ventanas retrospectivas y al suavizamiento. Los últimos cinco objetivos faltan porque su futuro no está contenido en la muestra. Aunque el pronóstico puede existir en esas fechas, no puede evaluarse todavía.

```
base = pd.concat([datos[[objetivo]], pred], axis=1)
print(base.isna().sum())
valida = base.notna().all(axis=1)
pos = np.flatnonzero(valida.to_numpy())
assert len(pos) > 100
assert np.all(np.diff(pos) == 1), (
    "Hay huecos interiores: revisar antes de continuar")
comun = base.loc[valida].copy()
assert np.isfinite(comun.to_numpy()).all()
assert comun.index.is_unique
assert comun.index.is_monotonic_increasing
print("Muestra comun:", len(comun))
print(comun.index.min(), comun.index.max())
```

Se eliminan filas únicamente según el objetivo y los siete pronósticos. Un `dropna()` sobre todas las variables originales podría reducir innecesariamente la muestra por columnas que no intervienen.

13. Evaluación preliminar en un bloque final

La separación formal con validación y prueba se desarrollará posteriormente, como señala el cronograma. Desde ahora evitamos resumir toda la serie como si fuera una evaluación final. Fijamos un bloque inicial del 70 % y un bloque final del 30 % de la muestra común. Los parámetros w y k se fijaron antes de revisar errores.

Estas reglas no tienen coeficientes que entrenar. El bloque inicial se reserva para desarrollo y comprobaciones; el bloque final ofrece una evaluación temporal preliminar. No sustituye la futura partición 70/15/15. Si se examinan repetidamente estos resultados para modificar reglas, este bloque deja de ser una prueba intacta.

13.1. Objetivos que atraviesan la frontera

Si T es la primera fecha del bloque final, una fila inicial cuyo objetivo termina en T o después contiene rendimientos del bloque final. Se excluye esa fila del bloque de desarrollo. Para $h = 5$, la condición conservadora es

$$\text{fecha}(t + 5) < T.$$

En modelos entrenados, esta precaución evita utilizar respuestas que aún no estaban disponibles al inicio del periodo de evaluación. Aquí la aplicamos para establecer el protocolo.

```
fin_objetivo = pd.Series(datos.index, index=datos.index)
fin_objetivo = fin_objetivo.shift(-h)
corte = int(np.floor(0.70 * len(comun)))
fecha_inicio = comun.index[corte]
inicial = comun.iloc[:corte].copy()
mascara = fin_objetivo.reindex(inicial.index) < fecha_inicio
desarrollo = inicial.loc[mascara].copy()
evaluacion = comun.iloc[corte:].copy()
assert len(desarrollo) > 0 and len(evaluacion) > 0
assert (fin_objetivo.loc[desarrollo.index]
        < fecha_inicio).all()
assert desarrollo.index.max() < evaluacion.index.min()
print("Desarrollo:", len(desarrollo))
print("Filas excluidas en frontera:",
      len(inicial) - len(desarrollo))
print("Evaluacion:", len(evaluacion))
```

No se borra una fila del bloque final por usar rendimientos históricos del bloque inicial: esa información era conocida. El protocolo permite actualizar los pronósticos cada día con la información recién observada, sin conocer rendimientos futuros.

14. Métricas de error

En n fechas comunes, definimos $e_t = Y_t - \widehat{Y}_t$. Entonces

$$\text{MAE} = \frac{1}{n}\sum_t |e_t|, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum_t e_t^2}, \qquad \text{Sesgo} = \frac{1}{n}\sum_t e_t.$$

MAE mide la magnitud promedio del error. RMSE penaliza con mayor fuerza los errores grandes. Ambos tienen las mismas unidades del objetivo. Un sesgo positivo indica subestimación promedio; uno negativo indica sobreestimación. Un sesgo cercano a cero no garantiza errores pequeños.

Por ejemplo, con errores $0.002, -0.001, 0.003$,

$$\text{MAE} = 0.002, \quad \text{RMSE} = \sqrt{\frac{0.000014}{3}} \approx 0.002160, \quad \text{Sesgo} \approx 0.001333.$$

Un MAE de 0.002 equivale a 0.2 puntos porcentuales de volatilidad diaria, cuando los rendimientos están en decimales.

14.1. MAPE y objetivos pequeños

Si todos los objetivos son positivos,

$$\text{MAPE} = \frac{100}{n}\sum_t \left|\frac{Y_t - \widehat{Y}_t}{Y_t}\right|.$$

Cuando $Y_t = 0$, MAPE no está definida; cuando es muy pequeño, puede dominar la comparación. Por ello MAE y RMSE son las métricas principales. No agregaremos una constante al denominador ni descartaremos ceros sin informar. Si hay un objetivo cero, se reporta MAPE como faltante para toda la muestra y se indica el número de ceros.

15. Cálculo de la tabla comparativa

```
def metricas(y, p):
    y = np.asarray(y, dtype=float)
    p = np.asarray(p, dtype=float)
    assert len(y) == len(p) and len(y) > 0
    assert np.isfinite(y).all() and np.isfinite(p).all()
    e = y - p
    mape = (100 * np.mean(np.abs(e / y))
            if np.all(y > 0) else np.nan)
    return {"N": len(y),
            "MAE": np.mean(np.abs(e)),
            "RMSE": np.sqrt(np.mean(e**2)),
            "Sesgo": np.mean(e),
            "MAPE_pct": mape,
            "Objetivos_cero": int(np.sum(y == 0))}

```

```
filas = []
for m in modelos:
    fila = metricas(evaluacion[objetivo], evaluacion[m])
    filas.append({"Modelo": m, **fila})
tabla = pd.DataFrame(filas).set_index("Modelo")
assert tabla["N"].nunique() == 1
assert (tabla["RMSE"] + 1e-12 >= tabla["MAE"]).all()
rmse_ref = tabla.loc["HV5", "RMSE"]
tabla["Mejora_RMSE_pct_vs_HV5"] = (
    100 * (1 - tabla["RMSE"] / rmse_ref)
    if rmse_ref > 0 else np.nan)
print(tabla.sort_values("RMSE"))
```

La mejora relativa positiva indica menor RMSE que HV5; una negativa indica mayor RMSE. Si HV5 tiene error cero, esa mejora relativa no está definida. Ordenar la tabla describe lo sucedido en este bloque; no permite afirmar superioridad universal ni seleccionar libremente un ganador para una prueba futura.

16. Gráficas e interpretación

Primero se compara HV5 con MA5 para mantener legible la figura. Las fechas del eje corresponden al origen del pronóstico; el objetivo ubicado en t describe las cinco sesiones posteriores.

```
fig, ax = plt.subplots(figsize=(12, 4))
for c, etiqueta in [(objetivo, "Volatilidad futura (5 sesiones)"),
                    ("HV5", "Historica HV5"),
                    ("MA5", "Media movil MA5")]:
    ax.plot(evaluacion.index, evaluacion[c],
            label=etiqueta, linewidth=1)
ax.set(xlabel="Fecha de origen del pronostico",
       ylabel="Volatilidad diaria (decimal)",
       title="Modelos base: evaluacion temporal preliminar")
ax.legend()
fig.tight_layout()
fig.savefig(salida / "01_observado_predicho.png", dpi=180)
plt.show()

fig, ax = plt.subplots(figsize=(8, 4))
tabla[["MAE", "RMSE"]].plot.bar(ax=ax)
ax.set(ylabel="Error (decimal)", xlabel="Modelo",
       title="Errores sobre las mismas fechas")
fig.tight_layout()
fig.savefig(salida / "02_metricas.png", dpi=180)
plt.show()

fig, ax = plt.subplots(figsize=(12, 3.5))
for m in ["HV5", "MA5"]:
    error = evaluacion[objetivo] - evaluacion[m]
    ax.plot(error.index, error, label=m, linewidth=0.8)
ax.axhline(0, color="black", linewidth=0.8)
```

```
ax.set(xlabel="Fecha de origen", ylabel="Observado - predicho",
       title="Errores: valores positivos indican subestimacion")
ax.legend()
fig.tight_layout()
fig.savefig(salida / "03_errores.png", dpi=180)
plt.show()
```

Al revisar las figuras se debe preguntar si el suavizamiento reduce oscilaciones, si retrasa la respuesta a picos y si existen periodos sostenidos de subestimación. Una curva visualmente más suave no necesariamente tiene menor error.

17. Solapamiento de ventanas futuras

Y_t usa $r_{t+1}, \ldots, r_{t+5}$ y Y_{t+1} usa $r_{t+2}, \ldots, r_{t+6}$: comparten cuatro rendimientos. Los errores sucesivos pueden estar correlacionados y no deben tratarse automáticamente como observaciones independientes.

Las métricas calculadas son resúmenes descriptivos válidos del bloque. No se utilizarán pruebas de significancia ni intervalos construidos con una hipótesis de independencia sin justificarla. Como ejercicio posterior puede evaluarse cada quinta sesión, documentando el punto de inicio; eso elimina el solapamiento de los objetivos seleccionados, pero no garantiza independencia de la serie financiera.

18. Exportación y trazabilidad

Se conserva la base completa, incluidos los pronósticos de las últimas cinco fechas sin objetivo observado. Se exporta además la muestra exacta evaluada y una tabla de periodos.

```
base.to_csv(salida / "SPY_2015_2025_modelos_base.csv")
evaluacion.to_csv(salida / "muestra_evaluacion.csv")
tabla.to_csv(salida / "metricas_modelos_base.csv")
periodos = pd.DataFrame([
    {"Bloque": "desarrollo", "N": len(desarrollo),
     "Inicio": desarrollo.index.min(),
     "Fin": desarrollo.index.max()},
    {"Bloque": "evaluacion_preliminar", "N": len(evaluacion),
     "Inicio": evaluacion.index.min(),
     "Fin": evaluacion.index.max()}
])
periodos.to_csv(salida / "periodos.csv", index=False)

leida = pd.read_csv(salida / "muestra_evaluacion.csv",
                    index_col=0, parse_dates=True)
assert leida.index.equals(evaluacion.index)
assert leida.columns.tolist() == evaluacion.columns.tolist()
np.testing.assert_allclose(
    leida.to_numpy(), evaluacion.to_numpy(),
    rtol=1e-7, atol=1e-10)
```

El notebook deberá registrar las versiones de Python, pandas, NumPy y Matplotlib, los parámetros utilizados y las fechas de cada bloque. No se presentan resultados numéricos reales en estos apuntes: deberán obtenerse al ejecutar el notebook con la base del proyecto.

19. Productos esperados y conexión con la Semana 10

1. Notebook `09_modelos_base.ipynb` con los bloques ejecutados en orden.
2. Siete series de pronósticos documentadas: HV5, HV10, HV20, HV30, MA5, MA10 y MA20.
3. Base completa, muestra evaluada, tabla de métricas y registro de periodos.
4. Tres figuras con ejes y unidades explícitos.
5. Verificaciones de causalidad, alineación, faltantes y comparación sobre fechas comunes.
6. Conclusión preliminar con resultados obtenidos por la estudiante.

Semana 10. Se introducirá EWMA, que asigna pesos exponenciales a observaciones recientes. Sus pronósticos deberán compararse con estas referencias conservando el mismo objetivo, las mismas unidades y un protocolo temporal explícito. La definición de media móvil usada aquí se mantendrá documentada para evitar confundir suavizamiento de volatilidades con suavizamiento de varianzas.